

The Program in Action

When you run *numeralsystems.py*, the output will look like this:

Numeral System Counters, by Al Sweigart al@inventwithpython.com

```
--snip--
Enter the starting number (e.g. 0) > 0
Enter how many numbers to display (e.g. 1000) > 20
DEC: 0   HEX: 0   BIN: 0
DEC: 1   HEX: 1   BIN: 1
DEC: 2   HEX: 2   BIN: 10
DEC: 3   HEX: 3   BIN: 11
DEC: 4   HEX: 4   BIN: 100
DEC: 5   HEX: 5   BIN: 101
DEC: 6   HEX: 6   BIN: 110
DEC: 7   HEX: 7   BIN: 111
DEC: 8   HEX: 8   BIN: 1000
DEC: 9   HEX: 9   BIN: 1001
DEC: 10  HEX: A   BIN: 1010
DEC: 11  HEX: B   BIN: 1011
DEC: 12  HEX: C   BIN: 1100
DEC: 13  HEX: D   BIN: 1101
DEC: 14  HEX: E   BIN: 1110
DEC: 15  HEX: F   BIN: 1111
DEC: 16  HEX: 10  BIN: 10000
DEC: 17  HEX: 11  BIN: 10001
DEC: 18  HEX: 12  BIN: 10010
DEC: 19  HEX: 13  BIN: 10011
```

How It Works

You can get the binary and hexadecimal representations of a number in Python by calling the `bin()` and `hex()` functions, respectively:

```
>>> bin(42)
'0b101010'
>>> hex(42)
'0x2a'
```

Convert these strings back into decimal integers by calling `int()` and supplying the base to convert from, like so:

```
>>> int('0b101010', 2)
42
>>> int('0x2a', 16)
42
```

Keep in mind that the binary and hexadecimal “numbers” returned by `bin()` and `hex()` are actually string values: `bin(42)` returns the string `'0b101010'` and `hex(42)` returns the string `'0x2a'`. In programming, it is convention to add a `0b` prefix to binary numbers and `0x` prefix to hexadecimal numbers.